진행 안내
11주차 / v5

LAB09 지금 날씨 보여줘

제공 코드 해설 → 날씨 조회 → GPS → 사진 등록 완성 → AI 음성 설명 → 사진 넘기기

> **목표:** 모바일 장치의 결과를 상태에 저장하고, 이미 제공된 화면에 반영한다. 사진만 등록하는 앱을 먼저 완성한 뒤 음성 설명을 추가한다.

이번 실습의 두 가지 기준

날씨를 표시하는 UI와 모델, HTTP 통신은 제공한다. **Region 선택, 기능 호출, 상태 변경, 버튼 연결**을 직접 작성한다. `_region`은 starter에 없고 S01에서 처음 선언한다.

새 기능은 **버튼 표시 → Hot Reload → 함수 작성 → 버튼 연결 → 실행 확인**으로 진행한다. S03에는 녹음 버튼, 메모 안내, `_description`이 전혀 없다.

진행 지도

구간	학생이 완성할 내용	완료 확인
제공 코드 읽기	앱 구조 → 모델 → 서비스 → UI	입력하지 않고 전체 연결을 이해
S00	준비된 starter 실행	기본 배경, --°, 비활성 새로고침
S01	_region 선언, 날씨 조회 함수 연결	버튼을 누르면 실제 날씨 표시
S02	Manifest 위치 권한, 현재 위치 조회	기기 좌표로 같은 날씨 함수 호출
S03	사진 선택, 기록 반환, 홈 배경 연결	음성 없이 내 사진 배경 완성
S04	녹음, AI 설명, 기존 등록 기능 확장	메모가 있을 때만 홈에 표시
S05	여러 사진을 PageView로 표시	사진, 설명, 시각이 함께 전환
S06 선택	지역명 버튼과 BottomSheet	다른 지역 날씨 조회

문서를 읽는 표시

오른쪽 상단 박스는 **지금 열 파일**, 코드 위의 경로 띠는 **해당 코드의 수정 파일**이다. 흰색 코드는 제공 코드 또는 기존 문맥이며, 노란색은 이번 절차에서 추가하거나 바꾸는 부분이다.

> **설명 메모:** 이 문서는 v5 소스와 한 묶음이다. v4의 단계 번호나 starter를 섞지 않는다. 코드 옆 주석은 설명용이며 학생이 전부 타이핑할 필요는 없다. teacher 폴더와 tool 폴더는 사용하지 않는다.

읽기 전용
`lib/models/weather_models.dart`

제공 코드 읽기 1 | 실행 구조와 Region

> **목표:** 먼저 제공된 파일의 역할을 살펴보고, 지역을 표현하는 객체가 화면과 API를 어떻게 연결하는지 읽는다. 아직 코드를 수정하지 않는다.

실행은 **lib/main.dart → WeatherLabApp → WeatherHomePage** 순서다. `CreatePhotoPage`는 준비되어 있지만 S03 전에는 열리지 않는다.

파일	제공 역할
lib/main.dart	앱 시작, MaterialApp과 테마
models/weather_models.dart	Region, WeatherInfo, WeatherRecord
services/weather_service.dart	지역 좌표 → 날씨 API → WeatherInfo
services/ai_speech_service.dart	녹음 파일 경로 → AI 설명 String
screens/weather_home_page.dart	날씨 표시 UI와 표시용 State
screens/create_photo_page.dart	기본 미리보기와 비활성 사진 선택 버튼

파일 `lib/models/weather_models.dart`

제공 | Region 정의와 준비된 지역 목록

```
class Region {
  final String id; // 앱 안에서 지역을 구분하는 값
  final String name; // 화면에 표시할 지역명
  final double latitude; // 날씨 조회에 사용할 위도
  final double longitude; // 날씨 조회에 사용할 경도

  const Region(this.id, this.name, this.latitude, this.longitude);
}

// Region 목록: 위치 좌표는 제공 데이터이다.
const regions = <Region>[
  Region('seoul', '서울', 37.5665, 126.9780),
  Region('daejeon', '대전', 36.3504, 127.3845),
  Region('cheonan', '천안', 36.8151, 127.1139),
  Region('busan', '부산', 35.1796, 129.0756),
  Region('jeju', '제주', 33.4996, 126.5312),
];
```

`id`는 **앱 안에서 지역과 사진을 묶는 식별자**이다. Open-Meteo의 지역 코드가 아니다. `name`은 화면에 표시하고, `latitude`와 `longitude`는 날씨 조회에 사용한다.

> **설명 메모:** `regions.first`는 첫 번째 지역 객체이다. 이를 화면의 선택 지역으로 저장할 `_region`은 아직 없다. S01에서 학생이 직접 선언한다. `current`는 S02에서 사용할 실습용 ID이며 여러 실제 지역을 구분하는 행정코드가 아니다.

읽기 전용
`lib/models/weather_models.dart`

제공 코드 읽기 2 | WeatherInfo와 getter

> **목표:** API가 준 값과 그 값에서 계산하는 표시용 값을 구분한다. 모델과 배경 선택 코드는 수정하지 않는다.

파일 `lib/models/weather_models.dart`

제공 | 날씨 데이터와 두 getter

```
class WeatherInfo {
  final double temperature;
  final int humidity;
  final double windSpeed;
  final int weatherCode; // Open-Meteo의 숫자형 날씨 코드
  final DateTime observedAt; // 날씨 데이터의 기준 시각

  const WeatherInfo({
    required this.temperature,
    required this.humidity,
    required this.windSpeed,
    required this.weatherCode,
    required this.observedAt,
  });

  // getter: weatherCode를 한국어 문구로 계산한다.
  String get condition {
    if (weatherCode >= 71 && weatherCode <= 77) return '눈';
    if (weatherCode >= 51 && weatherCode <= 67 ||
        weatherCode >= 80 && weatherCode <= 82 ||
        weatherCode >= 95) return '비';
    if (weatherCode >= 2) return '흐림';
    return '맑음';
  }

  // condition에 맞는 기본 배경 파일 경로를 고른다.
  String get backgroundAsset {
    if (condition == '눈') return 'assets/images/weather/snow.jpg';
    if (condition == '비') return 'assets/images/weather/rain.jpg';
    if (condition == '흐림') return 'assets/images/weather/cloudy.jpg';
    return 'assets/images/weather/clear.jpg';
  }
}
```

`weatherCode`는 Open-Meteo 응답의 `weather_code` 숫자를 저장한다. 제공 모델은 이 숫자를 **맑음, 흐림, 비, 눈으로 간단히 묶어** 표시한다. 상세한 기상 분류를 모두 구현한 코드는 아니다.

읽는 값	의미
weather.temperature	API에서 받은 기온
weather.condition	weatherCode로 계산한 한국어 날씨 문구
weather.backgroundAsset	condition에 맞춰 선택한 기본 이미지 경로
weather.observedAt	API 날씨 데이터의 기준 시각

> **설명 메모:** `get condition`은 메서드처럼 로직을 작성하지만 `weather.condition`으로 읽는 getter이다. 괄호를 붙이지 않으며, 별도의 문자열 State에 중복 저장하지 않는다. 예를 들어 제공 코드에서 61은 비로 분류되고 rain.jpg를 선택한다.

읽기 전용
`lib/models/weather_models.dart`

제공 코드 읽기 3 | 기록 모델과 서비스

> **목표:** 사진 기록은 어떤 값을 묶는지, 제공 서비스는 무엇을 받아 무엇을 돌려주는지 미리 읽는다.

파일 `lib/models/weather_models.dart`

제공 | WeatherRecord의 필드 발췌

```
class WeatherRecord {
  final String regionId; // Region.id와 비교할 값
  final String imagePath; // 기기에 있는 이미지 파일 경로
  final String description; // S03은 빈 문자열, S04부터 AI 설명
  final bool fromCamera;
  final double? latitude;
  final double? longitude;
  final DateTime? capturedAt; // 촬영 시각, 모르면 null
  final DateTime createdAt; // 앱에 기록을 등록한 시각
```

S03에서는 `description`에 빈 문자열을 넣어 **사진만 등록**한다. S04부터 AI 결과를 넣는다. 필드가 모델에 있다는 이유로 메모 UI까지 미리 만들지는 않는다.

`regionId`는 사진을 현재 조회 지역과 비교할 값이다. `capturedAt`은 촬영 시각, `createdAt`은 등록 시각이다. 갤러리 사진의 원래 촬영 시각과 장소를 모르면 null을 사용한다.

파일 `lib/models/weather_models.dart`

제공 | 하단에 표시할 날짜 문자열

```
String get sourceLabel {
  if (!fromCamera || capturedAt == null) {
    return '갤러리 사진 (촬영 시각 미확인)';
  }
  final time = capturedAt!;
  final month = time.month.toString().padLeft(2, '0');
  final day = time.day.toString().padLeft(2, '0');
  final hour = time.hour.toString().padLeft(2, '0');
  final minute = time.minute.toString().padLeft(2, '0');
  return '${time.year}/$month/$day $hour:$minute';
}
```

서비스의 입력과 출력

호출	받는 값과 돌려주는 값
WeatherService().fetch(region)	Region의 좌표로 조회 → Future<WeatherInfo>
AiSpeechService().cleanSpeech(path)	녹음 파일 경로로 요청 → Future<String>

lib/services/weather_service.dart는 HTTP GET과 JSON 변환을, **lib/services/ai_speech_service.dart**는 파일 읽기와 AI HTTP 요청을 맡는다. 화면은 `await`로 받은 결과만 사용한다.

> **설명 메모:** AI 통신은 기존 제공 파일을 유지한다. 이미 별도로 수정한 프롬프트, 응답 처리, API 키 설정은 덮어쓰지 않는다. 기본 패키지에는 실제 키가 아닌 자리표시자가 있으며, 키가 포함된 파일은 배포하지 않는다.

읽기 전용
`lib/screens/weather_home_page.dart`

제공 코드 읽기 4 | 완성된 날씨 표시 UI

> **목표:** 기온, 날씨, 습도, 풍속, 기준 시각과 배경은 이미 연결되어 있다. 이 표시 코드는 수업 중 다시 타이핑하지 않는다.

파일 `lib/screens/weather_home_page.dart`

제공 | 표시용 State와 읽기용 이름

```
WeatherInfo? _weather; // 조회 결과, 처음에는 null
bool _loading = false; // 요청 중 표시용 상태

// build() 안의 읽기용 지역변수
final weather = _weather;
```

파일 `lib/screens/weather_home_page.dart`

제공 | 날씨에 따른 기본 배경

```
Image.asset(
  weather?.backgroundAsset ?? 'assets/images/weather/clear.jpg',
  fit: BoxFit.cover, // 영역을 채우며 일부는 잘릴 수 있음
),
```

파일 `lib/screens/weather_home_page.dart`

제공 | 값이 바뀌면 함께 바뀌는 표시 UI

```
Text(
  weather == null
      ? '--°' : '${weather.temperature.round()}°',
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 86,
    fontWeight: FontWeight.w200,
    color: Colors.white,
  ),
),
Text(
  weather?.condition ?? '새로고침을 눌러 주세요.',
  textAlign: TextAlign.center,
  style: const TextStyle(fontSize: 22, color: Colors.white),
),
if (weather != null)
  Text(
    '습도 ${weather.humidity}%  |  '
    '풍속 ${weather.windSpeed.round()}km/h',
    textAlign: TextAlign.center,
    style: const TextStyle(color: Colors.white70),
  ),
if (weather != null)
  Text(
    '기상 기준 ${weather.observedAt.hour.toString().padLeft(2, '0')}:'
    '${weather.observedAt.minute.toString().padLeft(2, '0')}',
    textAlign: TextAlign.center,
    style: const TextStyle(color: Colors.white70, fontSize: 12),
  ),
if (_loading) const LinearProgressIndicator(),
```

> **설명 메모:** 삼항 연산자, ?.와 ??, 문자열 보간은 복습으로 읽는다. `weather`는 새로운 State가 아니다. `_weather`에 결과를 저장하고 setState로 rebuild를 요청하면 이 UI가 새 값을 표시한다. `_region`과 조회 함수는 아직 제공하지 않았다.

읽기 전용
`lib/screens/create_photo_page.dart`

제공 코드 읽기 5 | 등록 화면과 UI 틀

> **목표:** 사진 선택 전의 화면 틀만 읽는다. 사진 경로, 선택 함수, 녹음 버튼과 메모 표시를 미리 준비하지 않는다.

파일 `lib/screens/create_photo_page.dart`

제공 | 홈에서 받을 데이터와 생성자

```
class CreatePhotoPage extends StatefulWidget {
  final Region region; // 홈에서 전달받은 지역
  final WeatherInfo? weather; // 날씨 조회 실패 시 null 가능

  const CreatePhotoPage({
    super.key,
    required this.region,
    required this.weather,
  });
```

State에서는 `widget.region`과 `widget.weather`로 전달받은 값을 읽는다. **Scaffold.body → SafeArea → ListView** 구조이므로 내용이 길어지면 스크롤하고 시스템 버튼 영역을 피한다.

파일 `lib/screens/create_photo_page.dart`

제공 | 선택 전에는 기본 이미지만 표시

```
Image.asset(
  widget.weather?.backgroundAsset ??
      'assets/images/weather/clear.jpg',
  fit: BoxFit.cover,
),
```

파일 `lib/screens/create_photo_page.dart`

제공 | Row 안의 카메라 버튼 발췌

```
OutlinedButton.icon(
  onPressed: null, // 아직 사진 선택 함수 없음
  icon: const Icon(Icons.camera_alt_outlined),
  label: const Text('카메라'),
),
```

갤러리도 같은 구조이며 icon과 label만 다르다. Row와 Expanded를 포함한 전체 배치는 제공되어 있다.

UI 요소	간단히 설명할 역할
Stack / Positioned	사진 위에 지역명과 현재 기온을 겹쳐 표시
Row / Expanded	두 버튼을 가로로 배치하고 같은 비율로 넓이 배분
OutlinedButton.icon	아이콘과 글자를 함께 받는 버튼 생성자
onPressed: null	실행할 동작을 아직 연결하지 않음

> **설명 메모:** 사진 안에 글자를 그려 파일로 저장하는 것이 아니라 화면에서만 겹친다. 초기 화면에는 `_imagePath`, `_pickPhoto()`, `_description`, 음성 안내 문구가 없다. 여기까지 제공 코드 읽기를 마친 뒤 S00을 시작한다.

실행 위치
프로젝트 루트

S00 | starter 실행과 준비

> **목표:** 제공 UI는 완성되어 있지만 데이터 조회는 연결되지 않은 시작 상태를 확인한다.

이 패키지는 **Flutter 소스 패키지**이다. `pubspec.yaml`이 있는 폴더를 연 뒤 Android 플랫폼 파일을 생성하고 실행한다. 기존 프로젝트에 적용할 때는 사용 중인 환경 설정과 AI 서비스 파일을 먼저 보존한다.

파일 프로젝트 루트 / 터미널

실행 준비

```
flutter create --platforms=android --project-name lab09_show_me_weather .
flutter pub get
flutter run
```

android/app/src/main/AndroidManifest.xml에는 처음에는 INTERNET만 있다. 위치 권한은 S02, 마이크 권한은 S04에서 직접 추가한다. 생성 후 권한 파일이 달라졌다면 `steps/S00_AndroidManifest.xml`을 기준으로 맞춘다.

> **실행 확인:** 상단에는 "조회 지역", 기온은 --°, 배경은 기본 맑음 이미지가 나온다. 새로고침 아이콘은 보이지만 동작하지 않는다. 자동 날씨 조회, GPS 요청, 사진 등록 버튼, 메모 UI는 아직 없다.

처음 학생이 작성할 것

S01에서 `_region` 선언 → `_loadWeather()` 작성 → 기존 새로고침 버튼 연결을 한다. 날씨 표시 UI나 배경 경로를 다시 수정하지 않는다.

제공 상태	처음부터 제공하지 않는 것
Region과 regions 목록	선택 지역을 담는 _region
WeatherInfo? _weather, bool _loading	날씨 조회를 실행하는 _loadWeather()
완성된 날씨 표시 UI	새로고침의 실제 onPressed 연결
등록 화면의 기본 사진과 비활성 버튼	사진 경로, 선택 함수, 메모 상태

실행 방식

일반 UI와 함수 수정은 **Hot Reload**로 확인한다. AndroidManifest 권한을 수정한 뒤에는 앱을 종료하고 **다시 빌드/실행**한다. 필드 추가 후 초기화 상태가 맞지 않으면 Hot Restart를 사용한다.

> **설명 메모:** 실기기를 우선한다. 위치, 카메라, 마이크의 기기 동작과 API 계정 권한은 수업 전 확인한다. 이 실습에서는 복잡한 장애 복구, 역지오코딩, Firebase, 실시간 음성 대화를 다루지 않는다.

수정 파일
`lib/screens/weather_home_page.dart`

S01 | 첫 번째 변화: 선택 지역의 날씨

S01-① 선택 지역을 처음 선언

State의 첫 필드에 `_region`을 추가한다. `_weather`, `_loading`과 날씨 UI는 그대로 둔다.

파일 `lib/screens/weather_home_page.dart`

S01-① | State 필드

```
class _WeatherHomePageState extends State<WeatherHomePage> {
  Region _region = regions.first; // 처음 조회할 지역
  WeatherInfo? _weather; // 제공: 조회 전에는 null
  bool _loading = false; // 제공: 조회 중 표시용 상태
```

헤더 Row의 첫 `const Text`를 아래로 교체한다. 문자열 대신 객체의 이름을 사용하므로 바깥 const는 제거하고 TextStyle에만 const를 둔다.

파일 `lib/screens/weather_home_page.dart`

S01-① | 헤더의 지역명 Text 교체

```
Text(
  _region.name, // 객체의 지역명 표시
  style: const TextStyle(color: Colors.white),
),
```

> **실행 확인:** Hot Reload 후 “조회 지역”이 “서울”로 바뀐다. 기온은 아직 --°이다. 이 변화가 객체를 화면에 처음 사용한 결과다.

S01-② 날씨 조회 함수 작성

파일 상단 모델 import 다음에 날씨 서비스를 추가한다. `_loadWeather()`는 State의 필드 선언 아래, `build()` 앞에 새로 작성한다.

파일 `lib/screens/weather_home_page.dart`

S01-② | 파일 상단 import

```
import '../models/weather_models.dart';
import '../services/weather_service.dart';
```

파일 `lib/screens/weather_home_page.dart`

S01-② | 새 메서드 전체

```
Future<void> _loadWeather() async {
  setState(() { _loading = true; _weather = null; });
  try {
    final result = await WeatherService().fetch(_region); // 지역 좌표로 조회
    if (!mounted) return; // 화면 종료 후 갱신 방지
    setState(() => _weather = result); // 결과 저장과 rebuild 요청
  } catch (_) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('날씨를 불러오지 못했습니다.')),
    );
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}
```

수정 파일
`lib/screens/weather_home_page.dart`

S01 | 버튼 연결 후 실제 값 확인

S01-③ 이미 있는 새로고침 버튼에 연결

헤더 Row의 `tooltip: '□□ □□□□'`인 IconButton에서 **onPressed 한 줄만 변경**한다. 기온 Text, getter, 습도와 풍속, 배경 Image.asset은 수정하지 않는다.

파일 `lib/screens/weather_home_page.dart`

S01-③ | 기존 새로고침 버튼

```
IconButton(
  onPressed: _loading ? null : _loadWeather,
  color: Colors.white,
  tooltip: '날씨 새로고침',
  icon: const Icon(Icons.refresh),
),
```

> **실행 확인:** Hot Reload 후 새로고침을 누른다. --°가 실제 기온으로 바뀌고 날씨, 습도, 풍속과 기준 시각이 나타난다. 조회한 날씨가 맑음이면 배경은 그대로일 수 있다.

학생에게 설명할 한 줄 연결

버튼 → WeatherService.fetch(_region) → await 결과 → _weather 저장 → 기존 UI rebuild

> **설명 메모:** `_loadWeather`는 함수를 전달하는 값이다. `_loadWeather()`로 쓰면 그 자리에서 호출하는 표현식이 된다. 이 단계에서는 initState 자동 조회를 추가하지 않아, 버튼을 눌러 생긴 변화를 직접 확인한다.

S01 완료 확인

직접 작성	이미 제공되어 손대지 않는 부분
Region _region = regions.first	Region 정의와 regions 목록
_loadWeather()	WeatherService의 HTTP/JSON
새로고침 onPressed	기온, 날씨, 습도, 풍속, 시각과 배경 표시

완료 참고본: **steps/S01_weather_home_page.dart**. 다음 단계는 고정 도시 좌표 대신 기기 좌표로 Region을 만드는 것이다.

수정 파일
`lib/screens/weather_home_page.dart`

S02 | 위치 버튼과 Android 권한

S02-① 위치 버튼부터 표시

홈 헤더 Row에서 `const Spacer()`와 기존 새로고침 IconButton 사이에 현재 위치 버튼을 넣는다.

파일 `lib/screens/weather_home_page.dart`

S02-① | Spacer 다음에 새 버튼 추가

```
const Spacer(),
IconButton(
  onPressed: null, // 현재 위치 기능은 다음에 연결
  color: Colors.white,
  tooltip: '현재 위치',
  icon: const Icon(Icons.my_location),
),
```

> **실행 확인:** Hot Reload 후 위치 아이콘이 보이지만 아직 동작하지 않는다. “기기의 현재 좌표를 조회해 날씨를 바꾸는 버튼”이라고 설명한다.

S02-② Manifest에 위치 권한 추가

파일 전환: android/app/src/main/AndroidManifest.xml. 기존 INTERNET 선언 바로 아래, `<application>`보다 앞에 두 줄을 추가한다. 기존 activity 설정은 유지한다.

파일 `android/app/src/main/AndroidManifest.xml`

S02-② | 기존 INTERNET 아래 두 줄 추가

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

> **설명 메모:** COARSE는 대략적 위치, FINE은 정밀 위치 접근을 선언한다. Manifest 선언만으로 허용이 완료되는 것은 아니며, 다음 코드에서 실행 중 권한을 요청한다.

> **실행 확인:** 앱을 종료한 뒤 flutter run으로 다시 빌드한다. Hot Reload만으로 Manifest 변경을 적용하지 않는다. 복구 파일은 checkpoints/AndroidManifest_cp02_location.xml이다.

새 파일
`lib/services/location_service.dart`

S02 | 위치 서비스와 권한 확인

S02-③ LocationService 새로 작성

새 파일: lib/services/location_service.dart. 다음 전체 코드를 입력한다. 기기의 위치 기능과 이 앱의 위치 권한은 별개의 조건이다.

파일 `lib/services/location_service.dart`

S02-③ | 새 파일 전체

```
import 'package:geolocator/geolocator.dart';

class LocationService {
  Future<Position> getCurrent() async { // 현재 위치를 비동기로 조회
    if (!await Geolocator.isLocationServiceEnabled()) { // 기기 설정
      throw Exception('기기 위치 서비스를 켜 주세요.');
    }
    var permission = await Geolocator.checkPermission(); // 이 앱의 권한
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission(); // 권한 요청
    }
    // denied: 현재 거부, 다시 요청할 수 있음
    // deniedForever: 앱에서 재요청 불가, 기기 설정에서 변경
    if (permission == LocationPermission.denied ||
        permission == LocationPermission.deniedForever) {
      throw Exception('위치 권한이 필요합니다.'); // 둘 다 조회 중단
    }
    return Geolocator.getCurrentPosition(); // 위치 조회 Future 반환
  }
}
```

검사	학생에게 설명할 핵심
isLocationServiceEnabled()	기기의 위치 기능이 켜져 있는가?
checkPermission()	이 앱의 위치 접근이 허용되어 있는가?
requestPermission()	현재 거부 상태일 때 사용자에게 권한 요청
denied 또는 deniedForever	둘 다 현재 조회 권한이 없으므로 중단

> **설명 메모:** denied는 요청 후 다시 거부되었을 수도 있고, deniedForever는 앱의 권한 요청 창으로 변경할 수 없는 상태다. 둘 중 하나라도 남아 있으면 조회를 진행하지 않는다. throw는 호출한 쪽의 catch로 오류를 전달한다.

`getCurrentPosition()`은 위치 조회 Future를 반환한다. 이 async 함수의 결과를 기다리는 호출자는 Position을 받는다. 수업에서는 현재 위치 좌표를 얻는 기능에 집중한다.

수정 파일
`lib/screens/weather_home_page.dart`

S02 | GPS 좌표로 기존 날씨 조회 재사용

S02-④ Region 생성과 버튼 연결

다시 열 파일: lib/screens/weather_home_page.dart. 상단에 서비스를 import하고 `_loadWeather()` 다음, `build()` 앞에 `_useCurrentLocation()`을 작성한다.

파일 `lib/screens/weather_home_page.dart`

S02-④ | 서비스 import

```
import '../services/weather_service.dart';
import '../services/location_service.dart';
```

파일 `lib/screens/weather_home_page.dart`

S02-④ | 현재 위치 처리 함수

```
Future<void> _useCurrentLocation() async {
  try {
    final position = await LocationService().getCurrent(); // 기기 좌표
    if (!mounted) return;
    setState(() {
      _region = Region(
        'current', '현재 위치', position.latitude, position.longitude,
      ); // 기기 좌표로 새로운 지역 객체 생성
    });
    await _loadWeather(); // 기존 날씨 조회 재사용
  } catch (_) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('현재 위치와 권한을 확인해 주세요.')),
    );
  }
}
```

S02-①의 `tooltip: '□□ □□'` 버튼에서 onPressed를 연결한다. 새로고침 버튼은 그대로 유지한다.

파일 `lib/screens/weather_home_page.dart`

S02-④ | 현재 위치 버튼

```
IconButton(
  onPressed: _useCurrentLocation,
  color: Colors.white,
  tooltip: '현재 위치',
  icon: const Icon(Icons.my_location),
),
```

> **실행 확인:** Hot Reload → 위치 버튼 → 권한 허용. 지역명이 “현재 위치”로 바뀌고 해당 좌표의 날씨를 조회한다. 권한을 거부하면 안내가 나타난다.

> **설명 메모:** 지역명을 도시나 동 이름으로 바꾸는 역지오코딩은 추가하지 않는다. current는 11주차용 구분값이다. 다른 실제 지역으로 이동한 기록까지 영구적으로 묶는 방식은 12주차 전에 다시 설계한다.

수정 파일
`lib/screens/weather_home_page.dart`

S03 | 사진만 등록하는 흐름부터 완성

> **목표:** 이 단계의 완료 조건은 촬영한 사진이 홈의 날씨 배경으로 나타나는 것이다. 음성 설명, 메모 안내와 녹음 버튼은 아직 없다.

S03-① 홈에 사진 등록 버튼 표시

홈의 하단 기본 배경 안내 다음 `SizedBox(height: 16)` 아래에 버튼을 추가한다.

파일 `lib/screens/weather_home_page.dart`

S03-① | 홈 하단의 새 버튼

```
const SizedBox(height: 16),
Align(
  alignment: Alignment.centerRight,
  child: FloatingActionButton(
    onPressed: null, // 먼저 버튼만 표시
    tooltip: '사진 등록',
    child: const Icon(Icons.add_a_photo_outlined),
  ),
),
```

> **실행 확인:** Hot Reload 후 오른쪽 아래 사진 등록 아이콘을 확인한다. 아직 눌러도 동작하지 않는다.

S03-② 등록 화면을 여는 함수 준비

파일 상단에 화면을 import하고 `_loading` 아래에 중복 열기 방지 변수를 추가한다. 함수는 다음 페이지에서 작성한다.

파일 `lib/screens/weather_home_page.dart`

S03-② | 화면 import

```
import '../services/location_service.dart';
import 'create_photo_page.dart';
```

파일 `lib/screens/weather_home_page.dart`

S03-② | 기존 필드 아래에 추가

```
bool _loading = false; // 제공: 조회 중 표시용 상태
bool _openingCreatePage = false; // 화면 중복 열기 방지
```

> **설명 메모:** 이 변수는 빠른 연속 클릭으로 등록 화면이 두 번 열리는 것을 막는다. 화면에 표시하는 값이 아니므로 이 플래그를 바꿀 때 setState는 필요 없다.

수정 파일
`lib/screens/weather_home_page.dart`

S03 | 등록 화면 열기

수정 위치: `_useCurrentLocation()` 다음, `build()` 앞에 `_openCreatePage()`를 작성한다. 현재 위치가 아닌 지역을 보고 있다면 먼저 기기 위치를 확인한다.

파일 `lib/screens/weather_home_page.dart`

S03-② | 등록 화면 열기 함수 전체

```
Future<void> _openCreatePage() async {
  if (_openingCreatePage) return; // 연속 클릭은 무시
  _openingCreatePage = true;
  try {
    if (_region.id != 'current') {
      await _useCurrentLocation();
      if (!mounted || _region.id != 'current') return;
    }
    await Navigator.push<void>(
      context,
      MaterialPageRoute(
        builder: (context) => CreatePhotoPage(
          region: _region, weather: _weather,
        ), // 생성자로 지역과 날씨 전달
      ),
    );
  } finally {
    _openingCreatePage = false; // 돌아오면 다시 열기 허용
  }
}
```

S03-①의 FloatingActionButton을 아래처럼 연결한다.

파일 `lib/screens/weather_home_page.dart`

S03-② | 기존 사진 등록 버튼의 콜백

```
child: FloatingActionButton(
  onPressed: _openCreatePage,
  tooltip: '사진 등록',
  child: const Icon(Icons.add_a_photo_outlined),
),
```

실행 확인: Hot Reload 후 버튼을 눌러 등록 화면을 연다. 기본 사진, 지역과 기온, 비활성 카메라/갤러리 버튼만 보인다. 메모 안내와 녹음 버튼은 없다. 뒤로 가기로 홈에 돌아올 수 있다.

설명 메모: 지금은 push<void>로 화면 이동만 확인한다. S03-⑧에서 사진 기록을 받을 때 push<WeatherRecord>로 바꾼다. 이 타입은 이동하는 화면 클래스가 아니라 돌아올 결과의 타입이다.

수정 파일
lib/screens/create_photo_page.dart

S03 | 등록 버튼 표시와 사진 선택 함수

S03-③ 등록 화면에 완료 버튼부터 추가

파일 전환: lib/screens/create_photo_page.dart. `ListView.children`의 카메라/갤러리 Row 바로 뒤, children을 닫는 대괄호 앞에 다음을 추가한다.

파일 lib/screens/create_photo_page.dart

S03-③ | Row 다음에 추가

```
const SizedBox(height: 22),
FilledButton(
  onPressed: null, // 사진 등록 동작은 다음에 연결
  child: const Text('현장 사진 등록'),
),
```

> **실행 확인:** 등록 버튼은 보이지만 아직 동작하지 않는다. 사진 선택 버튼과 함께 스크롤되는지 확인한다.

S03-④ 사진 경로와 선택 함수 처음 작성

파일 맨 위에 import를 추가하고 State에 사진 상태를 처음 선언한다. 빈 함수는 미리 두지 않는다.

파일 lib/screens/create_photo_page.dart

S03-④ | 파일 상단 import

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
```

파일 lib/screens/create_photo_page.dart

S03-④ | State의 첫 필드

```
class _CreatePhotoPageState extends State<CreatePhotoPage> {
  String? _imagePath; // 선택한 사진 경로
  bool _fromCamera = false; // 직접 촬영인지 구분
  DateTime? _capturedAt; // 갤러리 촬영 시각은 미확인
```

같은 State의 `build()` 앞에 `_pickPhoto()`를 작성한다.

파일 lib/screens/create_photo_page.dart

S03-④ | 사진 선택 함수 전체

```
Future<void> _pickPhoto(ImageSource source) async {
  final image = await ImagePicker().pickImage(
    source: source, maxWidth: 1200,
  );
  if (image == null || !mounted) return; // 취소 또는 화면 종료
  setState(() {
    _imagePath = image.path; // 선택 결과를 화면 상태에 저장
    _fromCamera = source == ImageSource.camera;
    _capturedAt = _fromCamera ? DateTime.now() : null;
  });
}
```

> **설명 메모:** 카메라의 capturedAt은 촬영 결과가 돌아온 시각의 근사값이다. 갤러리는 촬영 시각 미확인으로 null을 저장한다.

수정 파일
`lib/screens/create_photo_page.dart`

S03 | 선택 사진을 미리보기에 연결

S03-⑤ 기본 이미지와 파일 이미지 구분

사진 미리보기 `Stack.children`의 첫 `Image.asset(...)`만 아래로 교체한다. 그라데이션과 지역명, 기온은 그대로 둔다.

파일 `lib/screens/create_photo_page.dart`

S03-⑤ | Stack의 첫 이미지 교체

```
_imagePath == null
    ? Image.asset(
        widget.weather?.backgroundAsset ??
            'assets/images/weather/clear.jpg',
        fit: BoxFit.cover,
      )
    : Image.file(
        File(_imagePath!), fit: BoxFit.cover,
      ),
```

사진 아래의 “사진을 선택해 주세요.” Text도 현재 선택 상태를 읽게 바꾼다.

파일 `lib/screens/create_photo_page.dart`

S03-⑤ | 사진 아래 안내

```
Text(_imagePath == null ? '사진을 선택해 주세요.' : '사진 미리보기'),
```

S03-⑥ 갤러리부터 연결하고 카메라로 확장

Row 안에서 **갤러리 버튼**의 onPressed를 먼저 바꾸고 실행한다.

파일 `lib/screens/create_photo_page.dart`

S03-⑥ | 갤러리 버튼

```
OutlinedButton.icon(
  onPressed: () => _pickPhoto(ImageSource.gallery),
  icon: const Icon(Icons.photo_library_outlined),
  label: const Text('갤러리'),
),
```

> **실행 확인:** Hot Reload 후 갤러리에서 사진을 고르면 미리보기가 바뀐다. 다시 선택하다 취소하면 기존 사진이 유지된다. 등록 버튼은 아직 비활성이다.

이제 **카메라 버튼**의 onPressed도 연결한다. 같은 함수에 다른 source만 전달한다.

파일 `lib/screens/create_photo_page.dart`

S03-⑥ | 카메라 버튼

```
OutlinedButton.icon(
  onPressed: () => _pickPhoto(ImageSource.camera),
  icon: const Icon(Icons.camera_alt_outlined),
  label: const Text('카메라'),
),
```

> **실행 확인:** 카메라로 촬영하고 확인하면 같은 미리보기 영역에 사진이 나타난다.

수정 파일
`lib/screens/create_photo_page.dart`

S03 | 사진 기록을 홈으로 반환할 준비

S03-⑦ 사진만 담는 등록 함수

`_pickPhoto()` 다음, `build()` 앞에 `_saveRecord()`를 새로 작성한다. 아직 등록 버튼은 null로 유지하고, 다음 절차에서 홈의 수신 코드를 먼저 준비한다.

파일 `lib/screens/create_photo_page.dart`

S03-⑦ | 등록 함수 전체

```
void _saveRecord() {
  if (_imagePath == null) return; // 사진이 없으면 등록하지 않음
  final record = WeatherRecord(
    regionId: widget.region.id, // 홈과 같은 지역 식별자
    imagePath: _imagePath!,
    description: '', // S03은 사진만 등록
    fromCamera: _fromCamera,
    latitude: _fromCamera ? widget.region.latitude : null,
    longitude: _fromCamera ? widget.region.longitude : null,
    capturedAt: _capturedAt,
    createdAt: DateTime.now(), // 앱에 등록한 시각
  );
  Navigator.pop(context, record); // 현재 화면을 닫으며 결과 전달
}
```

description은 빈 문자열이다. 사진 등록에 음성 메모는 필요하지 않다. 화면에도 “설명 없음” 같은 자리 문구를 미리 넣지 않는다.

코드	짧은 설명
widget.region.id	생성자로 전달받은 지역과 기록 연결
_imagePath!	위의 null 검사 후 선택 경로 사용
latitude / longitude	카메라만 현재 좌표 기록, 갤러리는 null
Navigator.pop(context, record)	등록 화면을 닫으며 객체를 홈에 반환

> **설명 메모:** 다음 두 절차를 마친 뒤 등록을 실행한다. 지금 버튼을 먼저 연결하면 홈이 기록 결과를 받아 배경에 반영하는 부분이 아직 없다. 생성자 전달과 pop 결과 반환은 서로 다른 방향의 데이터 이동이다.

수정 파일
lib/screens/weather_home_page.dart

S03 | 홈에서 기록 수신과 사진 배경 연결

S03-⑧ 결과를 받고 목록에 추가

파일 맨 위에 dart:io를 추가한다. State의 _openingCreatePage 아래에 기록 목록을 둔다.

파일 lib/screens/weather_home_page.dart

S03-⑧ | 파일 맨 위에 추가

```
import 'dart:io';
```

파일 lib/screens/weather_home_page.dart

S03-⑧ | State의 필드 추가

```
final List<WeatherRecord> _records = []; // 실행 중에만 보관
```

try 안의 `await Navigator.push<void>(...)` 구간만 교체한다. GPS 검사와 finally는 유지한다.

파일 lib/screens/weather_home_page.dart

S03-⑧ | 화면 결과를 받는 구간 교체

```
final record = await Navigator.push<WeatherRecord>(
  context,
  MaterialPageRoute(
    builder: (context) => CreatePhotoPage(
      region: _region, weather: _weather,
    ),
  ),
);
if (record == null || !mounted) return; // 취소하면 결과 없음
setState(() => _records.insert(0, record)); // 최신 사진을 맨 앞에
```

build()의 첫 부분을 확장한다. 같은 지역의 최신 사진 한 장만 고른다.

파일 lib/screens/weather_home_page.dart

S03-⑧ | build 시작 부분

```
final weather = _weather; // State를 읽는 지역변수, 새 State가 아님
final photos = _records.where((r) => r.regionId == _region.id).toList();
final shownPhoto = photos.isEmpty ? null : photos.first; // 최신 한 장
```

Stack의 첫 Positioned.fill을 다음 코드로 교체한다.

파일 lib/screens/weather_home_page.dart

S03-⑧ | 파일 사진을 홈 배경에 표시

```
Positioned.fill(
  child: shownPhoto == null
      ? Image.asset(
          weather?.backgroundAsset ??
              'assets/images/weather/clear.jpg',
          fit: BoxFit.cover,
        )
      : Image.file(
          File(shownPhoto.imagePath), fit: BoxFit.cover,
        ),
),
```

수정 파일
`lib/screens/weather_home_page.dart`

S03 | 사진만 등록한 앱의 완성 확인

홈 하단의 기본 배경 안내 Text를 다음으로 교체한다. 사진이 있으면 촬영 시각, 없으면 기본 배경 안내를 보여준다. 메모 UI는 아직 없다.

파일 `lib/screens/weather_home_page.dart`

S03-⑧ | 홈 하단의 보조 정보

```
Text(
  shownPhoto?.sourceLabel ?? '기본 날씨 배경 (현장 사진 없음)',
  style: const TextStyle(color: Colors.white70, fontSize: 12),
),
```

S03-⑨ 등록 버튼을 연결하고 끝까지 확인

파일 전환: lib/screens/create_photo_page.dart. "현장 사진 등록" FilledButton의 onPressed를 바꾼다.

파일 `lib/screens/create_photo_page.dart`

S03-⑨ | 기존 등록 버튼의 콜백

```
FilledButton(
  onPressed: _imagePath == null ? null : _saveRecord,
  child: const Text('현장 사진 등록'),
),
```

> **실행 확인:** 등록 화면을 닫았다 다시 연다. 카메라로 촬영 → 사진 미리보기 확인 → 현장 사진 등록. **홈으로 돌아오며 내가 찍은 사진 위에 현재 날씨가 표시되어야 한다.** 음성 녹음 없이 이 흐름을 끝낸다.

S03의 종료 기준

확인	기대 결과
사진이 없을 때	등록 버튼 비활성
카메라 또는 갤러리 선택	미리보기 변경, 등록 가능
등록 버튼	화면 닫힘, 홈 배경이 선택 사진으로 변경
두 번째 사진 등록	목록에 누적되지만 현재는 최신 사진만 표시
메모와 녹음 UI	아직 없음. S04에서 새로 추가

> **설명 메모:** 사진이 바뀌어도 기온은 현재 조회한 날씨이다. 사진에서 기온을 추정하는 기능이 아니다. 등록은 메모리 목록에 넣는 것이며 서버 저장이나 파일의 영구 보관이 아니다.

복구: **checkpoints/create_photo_page_cp03_photo.dart**, **checkpoints/weather_home_page_cp03_photo.dart**를 각각 lib/screens의 원래 파일명으로 복사한다.

수정 파일
`lib/screens/create_photo_page.dart`

S04 | 완성된 사진 앱에 음성 설명 추가

> **목표:** 사진만 등록할 수 있는 S03을 유지하면서 음성 설명을 선택 기능으로 붙인다. 설명을 녹음하지 않아도 등록할 수 있다.

S04-① 설명 UI와 녹음 버튼부터 표시

수정 파일: lib/screens/create_photo_page.dart. 카메라/갤러리 Row 뒤부터 기존 등록 FilledButton 시작까지의 간격을 다음처럼 확장한다. 등록 버튼의 onPressed와 나머지 본문은 그대로 둔다.

파일 `lib/screens/create_photo_page.dart`

S04-① | 등록 버튼 앞에 새 UI 삽입

```
const SizedBox(height: 22),
const Text('마이크로 현장 날씨를 설명해 주세요.'),
const SizedBox(height: 12),
FilledButton.icon(
  onPressed: null, // 녹음 동작은 다음에 연결
  icon: const Icon(Icons.mic),
  label: const Text('설명 녹음하기'),
),
const SizedBox(height: 12),
FilledButton(
```

> **실행 확인:** Hot Reload 후 사진 선택 버튼과 등록 버튼 사이에 안내와 녹음 버튼이 보인다. 녹음 버튼은 아직 동작하지 않지만 기존 사진 등록은 계속 가능하다.

S04-② Manifest에 마이크 권한 추가

파일 전환: android/app/src/main/AndroidManifest.xml. 위치 권한 아래, `<application>` 앞에 RECORD_AUDIO 한 줄을 넣는다.

파일 `android/app/src/main/AndroidManifest.xml`

S04-② | 마이크 권한 한 줄 추가

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

> **실행 확인:** Hot Reload가 아니라 앱을 다시 빌드한다. 복구는 checkpoints/AndroidManifest_cp04_audio.xml을 사용한다.

수정 파일
`lib/screens/create_photo_page.dart`

S04 | 녹음 상태를 처음 추가

S04-③ import와 State

다시 열 파일: lib/screens/create_photo_page.dart. 파일 상단에 녹음, 임시 폴더, AI 서비스 import를 추가한다.

파일 `lib/screens/create_photo_page.dart`

S04-③ | 파일 상단 import

```
import 'package:image_picker/image_picker.dart';
import 'package:record/record.dart';
import 'package:path_provider/path_provider.dart';

import '../services/ai_speech_service.dart';
```

State의 `_capturedAt` 다음에 녹음 도구와 두 상태, 설명 문자열을 추가한다.

파일 `lib/screens/create_photo_page.dart`

S04-③ | 사진 상태 아래 추가

```
DateTime? _capturedAt; // 갤러리 촬영 시각은 미확인
final _recorder = AudioRecorder(); // 녹음 도구
bool _isRecording = false;
bool _isProcessing = false;
String _description = ''; // AI가 정리한 설명
```

필드	역할
_recorder	녹음 시작, 종료와 파일 생성
_isRecording	현재 녹음 중인지 기억
_isProcessing	AI 응답을 기다리는 동안 버튼 제어
_description	AI가 반환한 설명. 처음에는 빈 문자열

다음 페이지의 `_toggleRecording()`과 `dispose()`를 `_saveRecord()` 다음, `build()` 앞에 작성한다. 사진 선택 함수와 등록 함수는 유지한다.

> **설명 메모:** 설명을 직접 수정하는 입력창은 만들지 않는다. TextEditingController 없이 String State를 Text로 표시한다. AI API 내부 통신을 이 단계에서 다시 타이핑하지 않는다.

수정 파일
lib/screens/create_photo_page.dart

S04 | 녹음 종료 후 AI 설명 받기

S04-④ 녹음 함수와 정리 함수

수정 위치: `_CreatePhotoPageState`의 `_saveRecord()` 다음, `build()` 앞. 아래 메서드 두 개를 추가한다.

파일 lib/screens/create_photo_page.dart

S04-④ | _toggleRecording()과 dispose() 전체

```
Future<void> _toggleRecording() async {
  if (_isRecording) {
    final path = await _recorder.stop(); // 녹음 종료 후 파일 경로
    if (!mounted) return;
    setState(() => _isRecording = false);
    if (path == null) return;
    setState(() => _isProcessing = true);
    try {
      final result = await AiSpeechService().cleanSpeech(path);
      if (!mounted) return;
      setState(() => _description = result); // 설명을 화면에 반영
    } catch (_) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('AI 교정 실패: API 키와 연결을 확인하세요.')),
      );
    } finally {
      if (mounted) setState(() => _isProcessing = false);
    }
  } else {
    if (!await _recorder.hasPermission()) return; // 마이크 권한
    final dir = await getTemporaryDirectory(); // 임시 저장 위치
    await _recorder.start(
      const RecordConfig(
        encoder: AudioEncoder.wav, sampleRate: 16000, numChannels: 1,
      ),
      path: '${dir.path}/weather_note.wav',
    );
    if (mounted) setState(() => _isRecording = true);
  }
}
```

```
@override
void dispose() {
  _recorder.dispose(); // 화면 종료 시 녹음 도구 정리
  super.dispose();
}
```

설명 메모: 녹음 중이면 stop으로 WAV 경로를 받고 AI에 전달한다. 녹음 전이면 권한을 확인하고 start를 호출한다. sampleRate 16000은 초당 16,000개 샘플, numChannels 1은 모노이다. 화면을 떠날 때 녹음 도구를 정리한다.

AI 키는 **lib/services/ai_speech_service.dart**의 `openAiApiKey` 자리표시자를 개인 실습 환경에서 설정한다. 이미 개선한 해당 서비스 파일은 그대로 사용할 수 있다.

수정 파일
`lib/screens/create_photo_page.dart`

S04 | 버튼 상태와 설명 표시 연결

S04-⑤ 녹음 버튼과 등록 버튼 제어

S04-①에서 추가한 고정 안내 Text와 FilledButton.icon을 다음으로 교체한다. 기존 간격은 유지한다.

파일 `lib/screens/create_photo_page.dart`

S04-⑤ | 안내 또는 AI 설명 표시

```
Text(
  _description.isEmpty
      ? '마이크로 현장 날씨를 설명해 주세요.' : _description,
),
```

파일 `lib/screens/create_photo_page.dart`

S04-⑤ | 녹음 버튼의 상태별 동작

```
FilledButton.icon(
  onPressed: _isProcessing ? null : _toggleRecording,
  icon: Icon(_isRecording ? Icons.stop : Icons.mic),
  label: Text(_isProcessing
      ? 'AI가 문장을 정리하는 중...'
      : (_isRecording ? '녹음 종료' : '설명 녹음하기')),
),
```

그 아래 기존 등록 FilledButton은 사진이 없거나 녹음/처리 중일 때만 비활성화한다. 설명의 빈 문자열은 등록을 막는 조건이 아니다.

파일 `lib/screens/create_photo_page.dart`

S04-⑤ | 기존 등록 버튼 조건 확장

```
FilledButton(
  onPressed: _imagePath == null || _isRecording || _isProcessing
      ? null : _saveRecord,
  child: const Text('현장 사진 등록'),
),
```

> **실행 확인:** 녹음 시작 → 말하기 → 녹음 종료 → AI 처리 → 설명 표시를 확인한다. 이 시점에는 설명이 등록 화면에만 있다. 다음 절차에서 기존 사진 기록과 홈 화면에 연결한다.

S04-⑥ 기존 기록에 설명을 함께 전달

같은 파일 `_saveRecord()`의 WeatherRecord 생성에서 description 한 줄만 바꾼다. 새 등록 함수를 만들지 않는다.

파일 `lib/screens/create_photo_page.dart`

S04-⑥ | WeatherRecord 생성 인자

```
imagePath: _imagePath!,
description: _description, // 비어 있어도 사진 등록 가능
fromCamera: _fromCamera,
```

수정 파일
`lib/screens/weather_home_page.dart`

S04 | 홈에서 메모가 있을 때만 표시

파일 전환: lib/screens/weather_home_page.dart. 하단 촬영 시각 Text 바로 위에 설명 Text를 추가한다. 기존 sourceLabel 표시와 사진 등록 버튼은 유지한다.

파일 `lib/screens/weather_home_page.dart`

S04-⑥ | 설명은 위에, 촬영 정보는 아래에

```
if (shownPhoto != null && shownPhoto.description.isNotEmpty)
  Text(
    shownPhoto.description,
    maxLines: 3,
    overflow: TextOverflow.ellipsis,
    style: const TextStyle(fontSize: 17, color: Colors.white),
  ),
if (shownPhoto != null && shownPhoto.description.isNotEmpty)
  const SizedBox(height: 8),
Text(
  shownPhoto?.sourceLabel ?? '기본 날씨 배경 (현장 사진 없음)',
  style: const TextStyle(color: Colors.white70, fontSize: 12),
),
```

> **실행 확인:** 카메라 사진 선택 → 설명 녹음 → AI 결과 확인 → 등록. 홈 배경에 사진과 설명이 함께 나온다. 새 등록 화면에서 녹음하지 않고 등록하면 사진과 시각만 보인다.

음성 설명의 범위

S03에서 완성한 사진 등록 기능을 확장한 것이다. 기존 사진을 수정하는 화면은 아니므로, 새 등록 화면을 열어 등록하면 새 기록이 추가된다. 이전 사진에 자동으로 메모를 붙인다고 설명하지 않는다.

> **설명 메모:** AI가 반드시 일반 문장만 반환하거나 모든 자기 수정 표현을 정확히 교정한다고 보장하지 않는다. 이 자료는 음성 처리 API와 응답 정리 코드를 새로 개편하지 않는다. API 오류 시 사진만 등록하는 흐름은 유지한다.

복구: **checkpoints/create_photo_page_cp04_audio.dart**, **checkpoints/weather_home_page_cp04_audio.dart**. 마이크 권한 XML도 함께 맞춘다.

수정 파일
`lib/screens/weather_home_page.dart`

S05 | 여러 사진을 넘길 상태 준비

> **목표:** S03부터 목록에 쌓인 기록을 사용한다. 등록 기능을 새로 만들지 않고 최신 한 장 표시를 사진 넘기기로 확장한다.

S05-① 사진 번호와 현재 사진 계산

State의 `_records` 다음에 `_photoIndex`를 추가한다.

파일 `lib/screens/weather_home_page.dart`

S05-① | 사진 번호 필드

```
  final List<WeatherRecord> _records = []; // 실행 중에만 보관
  int _photoIndex = 0; // 보고 있는 사진의 번호
```

`build()`에서 기존 `photos` 계산은 유지하고 `shownPhoto = photos.first` 부분을 아래로 교체한다.

파일 `lib/screens/weather_home_page.dart`

S05-① | 선택된 사진 계산

```
  final shownIndex = _photoIndex < photos.length ? _photoIndex : 0;
  final shownPhoto = photos.isEmpty ? null : photos[shownIndex];
```

`_openCreatePage()`에서 기록을 받은 뒤의 setState를 다음으로 바꾼다. 새로 등록한 사진이 첫 장이 된다.

파일 `lib/screens/weather_home_page.dart`

S05-① | 등록 결과 반영 부분

```
  setState(() {
    _records.insert(0, record);
    _photoIndex = 0; // 새 사진 등록 후 첫 장
  });
```

`_useCurrentLocation()`의 Region 대입 다음에도 `_photoIndex = 0`을 넣는다.

파일 `lib/screens/weather_home_page.dart`

S05-① | GPS Region 변경 구간

```
  _region = Region(
    'current', '현재 위치', position.latitude, position.longitude,
  ); // 기기 좌표로 새로운 지역 객체 생성
  _photoIndex = 0; // 지역 변경 후 첫 장
```

> **설명 메모:** 이 번호는 0부터 시작한다. 화면에 표시할 때만 1을 더한다. 다음 절차에서 PageView와 연결하기 전에는 여전히 첫 사진을 보여준다.

수정 파일
`lib/screens/weather_home_page.dart`

S05 | PageView와 터치가 통과하는 장식

S05-② 배경의 최신 한 장 표시를 PageView로 교체

수정 위치: 홈 Stack.children의 첫 Positioned.fill. 사진이 없을 때의 기본 배경 분기는 유지한다.

파일 `lib/screens/weather_home_page.dart`

S05-② | 여러 사진 배경

```
Positioned.fill(
  child: photos.isEmpty
      ? Image.asset(
          weather?.backgroundAsset ??
              'assets/images/weather/clear.jpg',
          fit: BoxFit.cover,
        )
      : PageView.builder(
          key: ValueKey('${_region.id}-${photos.length}'),
          itemCount: photos.length,
          onPageChanged: (index) =>
              setState(() => _photoIndex = index),
          itemBuilder: (context, index) => Image.file(
            File(photos[index].imagePath), fit: BoxFit.cover,
          ),
        ),
),
```

바로 다음 그라데이션 Positioned.fill에서 **DecoratedBox 바깥에 IgnorePointer**를 추가한다. 기존 장식값은 바꾸지 않는다.

파일 `lib/screens/weather_home_page.dart`

S05-② | 기존 장식의 바깥 경계만 추가

```
const Positioned.fill(
  child: IgnorePointer( // 장식은 터치를 가로막지 않음
    child: DecoratedBox(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [
            Color(0x77000000),
            Color(0x00000000),
            Color(0xB8000000),
          ],
          stops: [0, 0.48, 1],
        ),
      ),
    ),
  ),
),
```

> **설명 메모:** onPageChanged가 사진 번호를 바꾸면 설명과 시각도 같은 사진을 읽는다. ValueKey는 지역이나 사진 수가 바뀔 때 PageView를 새로 시작하게 한다. IgnorePointer는 장식을 보이게 하되 사진 넘김 터치를 가로막지 않게 한다.

수정 파일
`lib/screens/weather_home_page.dart`

S05 | 사진 번호와 수평 넘김 확인

S05-③ 촬영 정보 아래에 사진 번호 추가

홈 하단 sourceLabel Text 바로 다음, 기존 `SizedBox(height: 16)` 앞에 사진 번호를 넣는다. 한 장일 때는 안내를 표시하지 않는다.

파일 `lib/screens/weather_home_page.dart`

S05-③ | 하단 보조 정보 다음에 추가

```
Text(
  shownPhoto?.sourceLabel ?? '기본 날씨 배경 (현장 사진 없음)',
  style: const TextStyle(color: Colors.white70, fontSize: 12),
),
if (photos.length > 1)
  Text(
    '사진 ${shownIndex + 1} / ${photos.length}  |  좌우로 넘기기',
    style: const TextStyle(color: Colors.white70),
  ),
```

> **실행 확인:** 현재 위치에서 사진 2장을 등록한다. 홈의 사진 영역을 좌우로 드래그하면 사진, 설명, 촬영 시각과 1/2 표시가 함께 바뀌어야 한다. 한 장뿐이면 넘길 다음 사진이 없다.

핵심 앱 완성

기능	실습 결과
날씨 조회	제공 UI에 API 결과 표시
기기 위치	권한 확인 후 좌표로 날씨 조회
사진 등록	카메라/갤러리 사진을 홈 배경에 반영
선택 음성 설명	녹음 종료 후 AI 결과를 기록과 홈에 연결
사진 넘기기	메모와 시간이 해당 사진에 맞게 함께 변경

> **설명 메모:** 앱을 재시작하면 메모리 목록은 사라진다. 12주차에는 사진 파일과 기록의 저장/공유를 연결한다. 자동 슬라이드쇼, 좋아요, 지역별 대표 사진 정렬은 이번 단계에 추가하지 않는다.

수정 파일
`lib/screens/weather_home_page.dart`

S06 선택 | 다른 지역 선택 버튼

S06-① 지역명을 아이콘 버튼으로 바꾸기

수정 위치: 홈 헤더 Row의 첫 지역명 Text. 먼저 TextButton.icon으로 교체하고 창을 열 동작은 null로 둔다.

파일 `lib/screens/weather_home_page.dart`

S06-① | 지역명 버튼 표시

```
TextButton.icon(
  onPressed: null, // 선택 창은 다음에 연결
  icon: const Icon(Icons.keyboard_arrow_down),
  label: Text(_region.name),
  style: TextButton.styleFrom(foregroundColor: Colors.white),
),
```

> **실행 확인:** Hot Reload 후 지역명 옆에 펼침 아이콘이 생긴다. 아직 선택 창은 열리지 않는다.

S06-② 지역 선택 창 작성

State의 `_openCreatePage()` 다음, `build()` 앞에 `_chooseRegion()`을 추가한다.

파일 `lib/screens/weather_home_page.dart`

S06-② | 선택한 Region을 받는 함수

```
Future<void> _chooseRegion() async {
  final picked = await showModalBottomSheet<Region>(
    context: context,
    builder: (sheetContext) => SafeArea(
      child: ListView(
        shrinkWrap: true, // 항목 수에 맞는 높이
        children: [
          for (final item in regions)
            ListTile(
              title: Text(item.name),
              onTap: () => Navigator.pop(sheetContext, item),
            ), // 선택한 Region을 결과로 전달
        ],
      ),
    ),
  );
  if (picked == null || !mounted) return; // 선택 없이 닫으면 null
  setState(() {
    _region = picked;
    _photoIndex = 0;
  });
  await _loadWeather(); // 선택 지역 날씨 재조회
}
```

> **설명 메모:** TextButton.icon은 아이콘과 label을 함께 받는 생성자다. showModalBottomSheet<Region>()는 선택 창을 열고 Future<Region?>을 돌려준다. pop으로 보낸 item을 await 뒤에서 받는 흐름은 사진 등록 결과 반환과 같다.

수정 파일
`lib/screens/weather_home_page.dart`

S06 선택 | 지역 전환과 최종 점검

S06-③ 지역 버튼에 함수 연결

S06-① TextButton.icon의 onPressed만 바꾼다. 나머지 스타일과 label은 유지한다.

파일 `lib/screens/weather_home_page.dart`

S06-③ | 기존 지역명 버튼

```
TextButton.icon(
  onPressed: _chooseRegion,
  icon: const Icon(Icons.keyboard_arrow_down),
  label: Text(_region.name),
  style: TextButton.styleFrom(foregroundColor: Colors.white),
),
```

> **실행 확인:** 지역 버튼 → 다른 도시 선택 → 창 닫힘 → 지역명과 날씨 변경. 다른 지역의 사진은 아직 없으므로 기본 배경으로 돌아간다. 사진 등록 버튼을 누르면 현재 기기 위치를 확인한 뒤 등록 화면을 연다.

최종 확인

테스트	기대 결과
위치 권한 허용 / 거부	현재 위치 조회 또는 간단한 오류 안내
사진 선택 후 다시 취소	기존 미리보기 유지
음성 없이 사진만 등록	홈 배경에 표시, 빈 메모 영역 없음
음성 설명 후 등록	사진 위에 설명, 아래에 촬영 정보
빠르게 + 버튼 연속 클릭	등록 화면 중복 열림 방지
사진 2장 좌우 드래그	사진과 설명, 시각, 번호가 함께 변경
등록 화면을 끝까지 스크롤	버튼이 시스템 내비게이션 영역과 겹치지 않음

> **설명 메모:** SafeArea는 등록 화면의 ListView 바깥에 있고, 등록 버튼은 그 목록 안에 있다. bottomNavigationBar를 사용하지 않는다. 실제 기기에서 겹침과 스크롤, 권한과 녹음을 확인한다.

자료 안내
마무리

복구 위치와 자료의 범위

작업 파일은 lib, 단계 참고본은 steps

lib/main.dart가 starter의 실행 진입점이다. 학생이 수정할 파일은 각 절차에 표시된 lib 하위 파일 또는 AndroidManifest이다. steps 파일을 그 자리에서 실행하는 방식이 아니다.

자료	복원 또는 확인 방법
steps/S00_*	처음 제공된 전체 파일
steps/S01_* ~ S06_*	그 단계에서 바뀐 파일만 있음. 앞 단계 변경을 누적 적용
checkpoints/*cp03_photo.dart	S03 사진 등록 완료. 화면 두 파일을 lib/screens에 복사
checkpoints/*cp04_audio.dart	S04 음성 설명 완료. 화면 두 파일을 lib/screens에 복사
AndroidManifest_cp02_location.xml	android/app/src/main/AndroidManifest.xml로 복사
AndroidManifest_cp04_audio.xml	마이크까지 포함한 동일 권한 경로로 복사
solution/	S06까지의 완성 코드. lib 안에 같은 상대 경로로 복사

권한 단계에서 진도를 놓쳤다면 **steps/S02_location_service.dart**도 lib/services/location_service.dart로 복원한다. solution/android의 Manifest는 lib가 아니라 프로젝트 android 경로에 복사한다.

그대로 유지한 범위

날씨/AI 서비스의 HTTP 처리, 모델의 날씨 분류, 패키지 버전은 v4 제공본을 유지했다. AI 응답 형식 문제를 해결하는 새 서비스 구현은 이 개정에 포함하지 않는다. 개인적으로 수정한 AI 서비스는 보존한다.

다음 주와의 연결

11주차는 메모리 목록과 임시 사진 파일을 사용한다. 12주차는 Firebase 저장과 공유, 13주차는 사진 적합성 검사와 Function Calling으로 확장한다. 실제 지역 식별은 current 임시 ID와 구분해 다시 설계한다.

확인한 것과 남은 확인

v5 starter에서 순서대로 코드 변경을 다시 적용하여 단계 완료본과 비교했다. PDF의 코드, 파일 경로와 단계 번호도 함께 대조했다. **Flutter SDK가 없는 환경이므로 flutter analyze, 앱 빌드, 실기기 동작과 실제 AI 호출은 수행하지 않았다.**

> **설명 메모:** 작성 기준: 첨부 Lab09 v4 소스와 진행서, 이후 대화에서 합의한 제공 범위와 사진 우선 실습 순서. 디자인 기준: 첨부 Lab04 통합 코딩 진행서 v4. PDF는 본문 10pt, 코드 8.3pt를 기준으로 사용한다.